TRABAJO FINAL 2ª Ev: Acceso a Datos – PL/SQL

Grupo: **2ºSI** Curso: 25/26
Fecha: 11/02/26

I. Instrucciones generales

Lee con atención todo el enunciado, de la primera a la última cláusula.

Es probable que en una primera o segunda lecturas no se entienda todo al detalle. Empieza trabajando aquellos puntos que entiendas mejor, y poco a poco irás entendiendo mejor los restantes.

Las secciones I y II son informativas. La sección III contiene la parte que debe implementarse de forma obligatoria. La sección IV es opcional.

II. Tablas

Nuestro escenario será la BD de una biblioteca

01. La tabla *obra* contiene la información de las obras literarias.

```
CREATE TABLE obra (
  id CHAR(5),
  titulo VARCHAR(100),
  anyo INTEGER,
  CONSTRAINT PK_obra PRIMARY KEY (id),
  CONSTRAINT NN_titulo CHECK (titulo IS NOT NULL)
);
```

02. La tabla *autor* contiene la información de los autores de las obras.

```
CREATE TABLE autor (
  id CHAR(4),
  nombre VARCHAR(30),
  apellidos VARCHAR(60),
  nacimiento DATE,
  CONSTRAINT PK_autor PRIMARY KEY (id),
  CONSTRAINT NN_nombre CHECK (nombre IS NOT NULL),
  CONSTRAINT NN_apellidos CHECK (apellidos IS NOT NULL)
);
```

03. La tabla autor_obra contiene la relación de autores y obras.

```
CREATE TABLE autor_obra (
  id_autor CHAR(4),
  id_obra CHAR(5),
  CONSTRAINT PK_autor_obra PRIMARY KEY (id_autor, id_obra),
  CONSTRAINT FK_autor_obra_id_autor FOREIGN KEY (id_autor) REFERENCES
  autor(id),
  CONSTRAINT FK_autor_obra_id_obra FOREIGN KEY (id_obra)
  REFERENCES obra(id)
);
```

04. La tabla *edicion* contiene la información de las ediciones publicadas de las obras.

```
CREATE TABLE edicion (
  id CHAR(6),
  id_obra CHAR(5),
  isbn VARCHAR(20),
  anyo INTEGER,
  CONSTRAINT PK_edicion PRIMARY KEY (id),
  CONSTRAINT NN_id_obra CHECK (id_obra IS NOT NULL),
  CONSTRAINT NN_isbn CHECK (isbn IS NOT NULL),
  CONSTRAINT FK_edicion FOREIGN KEY (id_obra) REFERENCES
obra(id)
);
```

05. La tabla *ejemplar* contiene la información de los ejemplares adquiridos por la biblioteca.

```
CREATE TABLE ejemplar (
  id_edicion CHAR(6),
  numero INTEGER,
  alta DATE,
  baja DATE,
  CONSTRAINT PK_ejemplar PRIMARY KEY (id_edicion, numero),
  CONSTRAINT FK_ejemplar FOREIGN KEY (id_edicion) REFERENCES
  edicion(id),
  CONSTRAINT NN_alta CHECK (alta IS NOT NULL)
);
```

06. Los identificadores serán alfanuméricos y generados de forma aleatoria. Por supuesto habrá que tener en cuenta antes de realizar la inserción que la PK no esté duplicada, caso de estarlo terminaremos de forma controlada el intento de inserción monitorizando por pantalla el error.

```
DECLARE

id5 CHAR(5);

BEGIN

  id5:= dbms_random.string('X', 5); -- insertar una obra

  INSERT INTO obra (id, titulo) VALUES (id5, 'Cinco semanas
en globo');

END;
```

07. Los ejemplares *de cada edición* serán numerados de forma consecutiva.

```
SELECT * FROM ejemplar WHERE id_edicion = 'FSDPC7';



id_edicion
numero           alta      baja

---------- ----- --------- ---------
               - -         -

                 2009/07/2 2013/08/0
FSDPC7         1 7         5

                 2009/07/2
FSDPC7         2 7         (NULL)

                 2013/08/2
FSDPC7         3 9         (NULL)
```

III. Operativa requerida

08. El alta de obras se implementará mediante una función *alta_obra*.

```
-----------------------------------------------------------------------
-----


-- efectos: alta de obra con título y año
-- retorno: id asignado, '-1' error

-----------------------------------------------------------------------
-----


FUNCTION alta_obra(p_titulo VARCHAR, p_anyo INTEGER DEFAULT
NULL)

RETURN VARCHAR IS

  -- escriba aquí las declaraciones

BEGIN

  -- escriba aquí el código

END;
```

09. El borrado de obras se implementará mediante una función *borrado_obra*.

```
-----------------------------------------------------------------------
-----


-- efectos: borrado de obra por id

-- retorno: 1 si borrado efectuado, 0 si no existe id, -1 error
-----------------------------------------------------------------------
-----


FUNCTION
  borrado_obra(p_id
  VARCHAR) RETURN
  INTEGER IS

  -- escriba aquí las declaraciones
BEGIN


  -- escriba aquí el código
END;
```

09.1 Auditoría del borrado de obras: Cada vez que se ejecute el borrado de una obra y para no perder datos, lo que querremos es que la obra borrada y todos los datos asociados a ella queden guardados en una/s tabla/s de histórico ocultas al usuario estándar, de modo que en cualquier momento se pueda auditar el borrado de obras recuperando todos sus datos y la fecha/hora en que fue borrada la obra.

```
-----------------------------------------------------------------------
-----


-- efectos: Auditoría del borrado de obra por id

-- retorno: No aplicable (TRIGGER)
-----------------------------------------------------------------------
-----


TRIGGER auditar_borrado_obra BEFORE/AFTER DELETE ON . . .

  -- escriba aquí las declaraciones
BEGIN


  -- escriba aquí el código
END;
```

10. El alta de autores se implementará mediante una función *alta_autor*.

```
-----------------------------------------------------------------------
-----


-- efectos: alta de autor con nombre, apellidos y nacimiento
-- retorno: id asignado, '-1' error

-----------------------------------------------------------------------
-----


FUNCTION alta_autor(p_nombre VARCHAR, p_apellidos VARCHAR,


                  p_nacimiento DATE DEFAULT NULL)

  RETURN VARCHAR IS


  -- escriba aquí las declaraciones
BEGIN


  -- escriba aquí el código
END;
```

11. El borrado de autores se implementará mediante una función *borrado_autor*.

```
-----------------------------------------------------------------------
-----


-- efectos: borrado de autor por id
-- retorno: 1 si borrado efectuado, 0 si no existe id, -1 error

-----------------------------------------------------------------------
-----


FUNCTION borrado_autor(p_id VARCHAR) RETURN INTEGER IS

  -- escriba aquí las declaraciones

BEGIN

  -- escriba aquí el código

END;
```

12. La vinculación de un autor a una obra se implementará mediante una función *vincular*.

```
-----------------------------------------------------------------------
-----


-- efectos: vincula un autor a una obra

-- retorno: 1 (sin error) , -1 error
-----------------------------------------------------------------------
-----


FUNCTION vincular(p_id_autor VARCHAR,
  p_id_obra VARCHAR) RETURN INTEGER IS

  -- escriba aquí las declaraciones
BEGIN


  -- escriba aquí el código
END;
```

13. La desvinculación de un autor de una obra se implementará mediante una función *desvincular*.

```
-----------------------------------------------------------------------
-----


-- efectos: desvincula un autor de una obra
-- retorno: 1 si desvinculación efectuada, 0 si no existe
   vínculo, -1 error

-----------------------------------------------------------------------
-----


FUNCTION desvincular(p_id_autor VARCHAR, p_id_obra VARCHAR)

RETURN INTEGER IS

  -- escriba aquí las declaraciones

BEGIN

  -- escriba aquí el código

END;
```

14. El alta de ediciones se implementará mediante una función *alta_edicion*.

```
-----------------------------------------------------------------------
-----


-- efectos: alta de edición con obra, isbn y año

-- retorno: id asignado, '-1' error
-----------------------------------------------------------------------
-----


FUNCTION alta_edicion(p_id_obra VARCHAR, p_isbn VARCHAR,

                    p_año INTEGER DEFAULT NULL)


  RETURN VARCHAR IS

  -- escriba aquí las declaraciones

BEGIN

  -- escriba aquí el código

END;
```

15. El borrado de ediciones se implementará mediante una función *borrado_edicion*.

```
-----------------------------------------------------------------
---------------


-- efectos: borrado de edición por id
-- retorno: 1 si borrado efectuado, 0 si no existe id

-----------------------------------------------------------------
---------------

FUNCTION borrado_edicion(p_id VARCHAR)

RETURN INTEGER IS

  -- escriba aquí las declaraciones

BEGIN

  -- escriba aquí el código

END;
```

16. El alta de ejemplares se implementará mediante una función *alta_ejemplar*.

```
-----------------------------------------------------------------
---------------


-- efectos: alta de ejemplar con id_edicion

-- retorno: número asignado, -1 si no existe id_edicion.
-----------------------------------------------------------------
---------------


FUNCTION
  alta_ejemplar(p_id_edicion
  VARCHAR) RETURN INTEGER IS

  -- escriba aquí las declaraciones
BEGIN


  -- escriba aquí el código
END;
```

17. La operación de alta de ejemplar no requiere especificar fecha de alta, porque será la fecha del sistema.

18. La operación de alta de ejemplar no requiere especificar el número, porque se calculará según los ejemplares de la misma edición ya existentes (si ya hay N, pues se dará de alta como N+1).

19. Para los ejemplares se distinguirá entre las operaciones de *borrado* y de *baja*, ya que tienen diferente significado y aplicación.

20. El borrado de ejemplares significa suprimir la fila correspondiente, y su aplicación habitual será revertir un alta errónea.

21. Sólo se podrá borrar un ejemplar si es el último de su serie, no tiene fecha de baja y además no han pasado más de 30 días desde la fecha de alta.

22. El borrado de ejemplares se implementará mediante una función *borrado_ejemplar*.

```
-----------------------------------------------------------------
---------------


-- efectos: borrado de ejemplar por id_edicion y número

-- retorno: 1 si borrado efectuado (cumpliendo cláusula 21),
--          0 borrado no efectuado (no existe el ejemplar
           indicado)

--          -1 borrado no efectuado (existe pero no cumple la
           cláusula 21)
-----------------------------------------------------------------
---------------


FUNCTION borrado_ejemplar(p_id_edicion VARCHAR,
  p_numero INTEGER) RETURN INTEGER IS

  -- escriba aquí las declaraciones
BEGIN


  -- escriba aquí el código
END;
```

23. La baja de un ejemplar consistirá en establecer la fecha de baja, y su aplicación será reflejar que éste ha sido retirado del sistema por deterioro o extravío.

24. La operación de baja no requiere especificar la fecha, porque se utilizará la del sistema.

25. Sólo se podrá dar de baja un ejemplar si no tiene fecha de baja.

26. La baja de ejemplares se implementará mediante una función *baja_ejemplar*.

```
-----------------------------------------------------------------
---------------


-- efectos: baja de ejemplar por id_edicion y número

-- retorno: 1 si baja efectuada (cumpliendo la cláusula 25)
--          0 baja no efectuada (no existe el ejemplar indicado)

--          -1 baja no efectuada (existe, pero no cumple la
            cláusula 25)
-----------------------------------------------------------------
---------------


FUNCTION baja_ejemplar(p_id_edicion VARCHAR,
  p_numero INTEGER) RETURN INTEGER IS

  -- escriba aquí las declaraciones
BEGIN


  -- escriba aquí el código
END;
```

IV. Operativa opcional (para llegar al 10, la parte obligatoria llega hasta el 7)

27. Definir una tabla con la información (nombre y apellidos) de los socios de la biblioteca.

```
CREATE TABLE socio (

  -- escriba aquí la definición de la tabla

);
```

28. Definir una tabla con la información de los préstamos.

```
CREATE TABLE prestamo (

  -- escriba aquí la definición de la tabla

);
```

29. Escriba una función llamada *apertura_prestamo* para abrir un préstamo de libro.

```
FUNCTION apertura_prestamo ( <parámetros-que-estime-oportunos> )

RETURN <tipo-de-retorno-que-estime-oportuno> IS

  -- escriba aquí las declaraciones

BEGIN

  -- escriba aquí el código

END;
```

30. Auditar el borrado de cualquier elemento de la BD tal y como realizamos en el punto 9.1 a través de TRIGGERS

30. Escriba una función llamada *cierre_préstamo* para cerrar un préstamo de libro.

```
FUNCTION cierre_prestamo ( <parámetros-que-estime-oportunos> )

RETURN <tipo-de-retorno-que-estime-oportuno> IS

  -- escriba aquí las declaraciones
BEGIN

  -- escriba aquí el código
END;
```

V. Formato de entrega

27. Deberá entregar un fichero ZIP con el formato APELLIDOS_NOMBRE.ZIP, con el código fuente PL/SQL que haya desarrollado en dos ficheros de SQL que contendrán los procedimientos y funciones realizados agrupados en Packages de PLSQL y un tercer y cuarto fichero que contendrá la parte de funciones y procedimientos opcional si se ha decidido llevarla a cabo. El resto de ficheros será la creación de tablas, la batería de pruebas que se ha realizado a nuestros procedimientos y/o funciones de los packets (ver punto 32, criterios de evaluación) y los triggers.

28. El primer fichero se llamará BIBLIOTK_PACKAGE.SQL, con el siguiente contenido.

```
CREATE OR REPLACE PACKAGE BiblioTK AS

  ... las funciones tal como deben aparecer en
CREATE PACKAGE END;
```

29. El segundo fichero se llamará BIBLIOTK_BODY. SQL, con el siguiente contenido.

```
CREATE OR REPLACE PACKAGE BODY BiblioTK AS

  ... las funciones tal y como deben aparecer en
CREATE PACKAGE BODY END;
```

30. El tercer fichero se llamará BIBLIOTK_ESQUEMA_PARTE2. SQL y contendrá la definición CREATE TABLE de las tablas *socio* y *préstamo* de la operativa opcional, si la implementa.

VI. Criterios de evaluación

31. Los ficheros entregados se ejecutarán en un entorno SQL para verificar que compilan y/o ejecutan correctamente.

32. Una vez introducidos en el entorno SQL, se ejecutará una batería invocaciones de las funciones del paquete BIBLIOTK implementado para verificar si cumplen las especificaciones.

33. Las cláusulas de la sección III (operativa requerida) califican por un 70% de la nota y su implementación es obligatoria (por tanto, prioritaria).

34. Las cláusulas de la sección IV (operativa opcional) califican por un 30% de la nota y su implementación es opcional (por tanto, secundaria).

VII. Fecha límite de entrega

Martes 10 de marzo de 2026

NO DUAL

Lunes 23 de marzo de 2026

DUAL.